368. Largest Divisible Subset - Explanation

Description
Given a set of distinct positive integers nums, return the largest subset answer such that every pair (answer[i], answer[j]) of elements in this subset satisfies:

answer[i] % answer[j] == 0, or
answer[j] % answer[i] == 0
If there are multiple solutions, return any of them.

Example 1:

Input: nums = [1,2,3]

Output: [1,2]
Explanation: [1,3] is also accepted.

Example 2:

Input: nums = [1,2,4,8]

Output: [1,2,4,8]
Constraints:

1 <= nums.length <= 1000
1 <= nums[i] <= 2,000,000,000
All the integers in nums are unique.

Prerequisites
Before attempting this problem, you should be comfortable with:

Dynamic Programming (Memoization) - The problem requires caching recursive results to avoid recomputation
Sorting - Elements must be sorted first to establish the divisibility chain property
Divisibility and Modulo Operations - Understanding when one number divides another evenly
Recursion - Top-down approaches use recursive function calls with state tracking

1. Dynamic Programming (Top-Down)

Intuition
A divisible subset has a special property: if we sort the numbers, then for any pair in the subset, the larger number must be divisible by the smaller one. This means if we sort the array and pick elements in order, we only need to check divisibility with the most recently picked element. We can use recursion with memoization to try including or skipping each number.

Algorithm
Sort the array in ascending order.
Define a recursive function dfs(i, prevIndex) that returns the largest divisible subset starting from index i, where prevIndex is the index of the last included element (or -1 if none).

At each index:
Option 1: Skip the current number.
Option 2: If no previous element exists or the current number is divisible by the previous one, include it and recurse.
Memoize results based on (i, prevIndex) to avoid recomputation.
Return the larger of the two options.

```
public class Solution {
    private List<Integer>[][] cache;

    public List<Integer> largestDivisibleSubset(int[] nums) {
        Arrays.sort(nums);
        int n = nums.length;
        cache = new ArrayList[n][n + 1];
        return dfs(0, -1, nums);
    }

    private List<Integer> dfs(int i, int prevIndex, int[] nums) {
        if (i == nums.length) return new ArrayList<>();
        if (cache[i][prevIndex + 1] != null) return cache[i][prevIndex + 1];

        List<Integer> res = dfs(i + 1, prevIndex, nums);

        if (prevIndex == -1 || nums[i] % nums[prevIndex] == 0) {
            List<Integer> tmp = new ArrayList<>();
            tmp.add(nums[i]);
            tmp.addAll(dfs(i + 1, i, nums));
            if (tmp.size() > res.size()) res = tmp;
        }

        cache[i][prevIndex + 1] = res;
        return res;
    }
}
```

Expand
Time & Space Complexity
Time complexity:
O(n^2)
Space complexity:
O(n^2)

2. Dynamic Programming (Top-Down) Space Optimized

Intuition
We can simplify the state by observing that we only need to track the starting index, not the previous index. For each starting position, we find the longest divisible subset that begins there. When building the subset from index i, we look at all later indices j where nums[j] is divisible by nums[i] and take the best result.

Algorithm
Sort the array in ascending order.
Define dfs(i) that returns the largest divisible subset starting at index i.
At each index i:
Initialize the result with just nums[i].
For each later index j where nums[j] % nums[i] == 0, recursively get the subset starting at j.
Prepend nums[i] to the best result and keep the longest.
Memoize results for each starting index.
Try all starting positions and return the longest subset found.

```
public class Solution {
    private List<Integer>[] cache;

    public List<Integer> largestDivisibleSubset(int[] nums) {
        Arrays.sort(nums);
        int n = nums.length;
        cache = new ArrayList[n];

        List<Integer> res = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            List<Integer> tmp = dfs(i, nums);
            if (tmp.size() > res.size()) {
                res = tmp;
            }
        }
        return res;
    }

    private List<Integer> dfs(int i, int[] nums) {
        if (cache[i] != null) return cache[i];

        List<Integer> res = new ArrayList<>();
        res.add(nums[i]);
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[j] % nums[i] == 0) {
                List<Integer> tmp = new ArrayList<>();
                tmp.add(nums[i]);
                tmp.addAll(dfs(j, nums));

                if (tmp.size() > res.size()) {
                    res = tmp;
                }
            }
        }
        return cache[i] = res;
    }
}
```

Expand
Time & Space Complexity

Time complexity:
O(n^2)
Space complexity:
O(n)

3. Dynamic Programming (Bottom-Up)
Intuition
We can convert the top-down approach to bottom-up. Processing from right to left, for each index we compute the longest divisible subset starting from that position. At each step, we check all later indices for valid extensions and build upon the precomputed results.

Algorithm
Sort the array in ascending order.
Create a DP array where dp[i] stores the longest divisible subset starting at index i.
Initialize each dp[i] with just the element at that index.
Iterate from right to left:
For each later index j, if nums[j] % nums[i] == 0, check if prepending nums[i] to dp[j] gives a longer subset.
Update dp[i] with the longest result.
Track the overall longest subset found.
Return the longest subset.

```
public class Solution {
    public List<Integer> largestDivisibleSubset(int[] nums) {
        Arrays.sort(nums);
        int n = nums.length;
        List<Integer>[] dp = new ArrayList[n];
        List<Integer> res = new ArrayList<>();

        for (int i = n - 1; i >= 0; i--) {
            dp[i] = new ArrayList<>();
            dp[i].add(nums[i]);

            for (int j = i + 1; j < n; j++) {
                if (nums[j] % nums[i] == 0) {
                    List<Integer> tmp = new ArrayList<>();
                    tmp.add(nums[i]);
                    tmp.addAll(dp[j]);

                    if (tmp.size() > dp[i].size()) {
                        dp[i] = tmp;
                    }
                }
            }
            if (dp[i].size() > res.size()) {
                res = dp[i];
            }
        }
```

```
        return res;
    }
}
```

Expand
Time & Space Complexity
Time complexity:
O(n^2)
Space complexity:
O(n)

4. Dynamic Programming (Top-Down) + Tracing
Intuition
Instead of storing entire subsets in the DP table (which uses extra memory), we can store just two values per index: the length of the longest subset starting there, and the next index in that subset. After computing all lengths, we trace through the indices to reconstruct the actual subset.

Algorithm
Sort the array in ascending order.
Create a DP array where dp[i] = [maxLen, nextIndex].
Define dfs(i) that returns the length of the longest subset starting at index i:
For each later index j where nums[j] % nums[i] == 0, compute the length via dfs(j) + 1.
Track which j gave the best length for tracing.
Find the starting index with the maximum length.
Reconstruct the subset by following the nextIndex pointers.

```
public class Solution {
    private int[][] dp;

    public List<Integer> largestDivisibleSubset(int[] nums) {
        Arrays.sort(nums);
        int n = nums.length;
        dp = new int[n][2];
        for (int i = 0; i < n; i++) {
            dp[i][0] = -1;
            dp[i][1] = -1;
        }

        int maxLen = 1, startIndex = 0;
        for (int i = 0; i < n; i++) {
            if (dfs(i, nums) > maxLen) {
                maxLen = dp[i][0];
                startIndex = i;
            }
        }

        List<Integer> subset = new ArrayList<>();
        while (startIndex != -1) {
```

```
            subset.add(nums[startIndex]);
            startIndex = dp[startIndex][1];
        }
        return subset;
    }

    private int dfs(int i, int[] nums) {
        if (dp[i][0] != -1) return dp[i][0];

        dp[i][0] = 1;
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[j] % nums[i] == 0) {
                int length = dfs(j, nums) + 1;
                if (length > dp[i][0]) {
                    dp[i][0] = length;
                    dp[i][1] = j;
                }
            }
        }
        return dp[i][0];
    }
}
```

Expand
Time & Space Complexity
Time complexity:
O(n^2)
Space complexity:
O(n)

5. Dynamic Programming (Bottom-Up) + Tracing
Intuition
This is the iterative version of the tracing approach. We process indices from left to right, looking backward for valid predecessors. For each position, we store the length of the longest subset ending there and a pointer to the previous index. This is similar to the classic Longest Increasing Subsequence pattern.

Algorithm
Sort the array in ascending order.
Create a DP array where dp[i] = [maxLen, prevIndex], initialized with length 1 and no predecessor.
For each index i, check all earlier indices j:
If nums[i] % nums[j] == 0 and extending from j gives a longer subset, update dp[i].
Track the index with the overall maximum length.
Reconstruct the subset by following the prevIndex pointers backward from the best ending index.

```
public class Solution {
    public List<Integer> largestDivisibleSubset(int[] nums) {
        Arrays.sort(nums);
```

```
        int n = nums.length;
        int[][] dp = new int[n][2]; // dp[i] = {maxLen, prevIdx}

        int maxLen = 1, startIndex = 0;
        for (int i = 0; i < n; i++) {
            dp[i][0] = 1;
            dp[i][1] = -1;
            for (int j = 0; j < i; j++) {
                if (nums[i] % nums[j] == 0 && dp[j][0] + 1 > dp[i][0]) {
                    dp[i][0] = dp[j][0] + 1;
                    dp[i][1] = j;
                }
            }

            if (dp[i][0] > maxLen) {
                maxLen = dp[i][0];
                startIndex = i;
            }
        }

        List<Integer> subset = new ArrayList<>();
        while (startIndex != -1) {
            subset.add(nums[startIndex]);
            startIndex = dp[startIndex][1];
        }
        return subset;
    }
}
```

Expand
Time & Space Complexity
Time complexity:
O(n^2)
Space complexity:
O(n)
Common Pitfalls
Forgetting to Sort the Array First
The divisibility property only guarantees transitivity when elements are sorted. If a divides b and b divides c, then a divides c, but this chain only works reliably when processing elements in ascending order. Skipping the sort step means you might miss valid subset extensions or incorrectly reject valid ones.

Checking Divisibility in the Wrong Direction
After sorting in ascending order, when comparing nums[i] and nums[j] where i < j, you must check if nums[j] % nums[i] == 0 (larger divisible by smaller). A common mistake is checking nums[i] % nums[j] == 0, which will always be false for distinct positive integers when nums[i] < nums[j].

Only Returning the Length Instead of the Actual Subset
The problem asks for the subset itself, not just its size. When using DP with length tracking only, you must also maintain a way to reconstruct the path, typically through parent pointers or by storing the actual subsets. Forgetting this reconstruction step means you can compute the correct length but cannot output the required elements.